

Chapter 4

UVM (Universal Verification Methodology)

Chapter Introduction

This chapter will describe in detail the architecture of UVM, UVM hierarchy, and discuss each hierarchical component (testbench, test, environment, agent, scoreboard, driver, monitor, sequencer, etc.) in detail. We will also go through two complete examples to solidify the concepts.

4.1 What Is UVM?

Ah, isn't this now the mother of all verification methodologies? Let us start with what UVM is and then delve into detail and examples.

UVM is a transaction-level methodology (TLM) designed for testbench development. It is a class library that makes it easy to write configurable and reusable code. You do need to understand the basic concepts of OOP (object-oriented programming), but the designers of UVM did all the hard work. They created the so-called class library whose components can be used to develop a testbench. You don't need to be an OOP expert. Reusable in the sense that once you put together the required code/infrastructure in place, using UVM class library, you will be able to carry forward that to the next project. Only the driver (in UVM agent), the scoreboard, and the basic transaction (sequence) and sequence library that contains tests need to change. With pure SystemVerilog, testbenches are written ad hoc since there is no coding standard there. These testbenches are then not quite reusable. The code is also hard to understand and maintain.

UVM is class based and communicates between these classes via transactions. This helps keep communication interface among class components separate from the implementation detail in the UVM agent driver. If you don't understand some of this, hold on. We'll go into the UVM hierarchy and examples to solidify the concepts. The UVM class library provides generic utilities, such as component

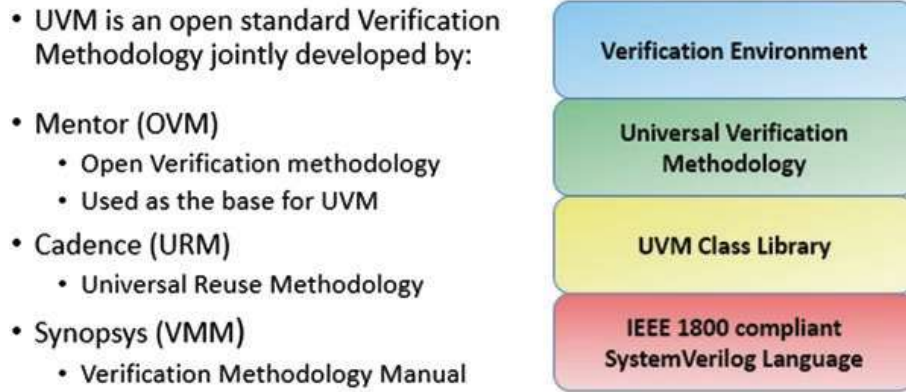


Fig. 4.1 Evolution of UVM

hierarchy, transaction library model (TLM), configuration database, etc., which enable the users to create virtually any structure they want for the testbench.

UVM provides a set of transaction-level communication interfaces and channels that you can use to connect components at the transaction level. The use of **TLM interfaces** isolates each component from changes in other components throughout the environment. When coupled with the phased, flexible-build infrastructure in UVM, TLM promotes reuse by allowing any component to be swapped for another, as long as they have the same interfaces. In addition, TLM provides the basis for easily encapsulating components into reusable components, called verification components, to maximize reuse and minimize the time and effort required to build a verification environment.

But I am getting carried away here. Let's see how UVM came to being.

As noted in Fig. 4.1, there were three competing methodologies in the industry. Open Verification Methodology (OVM) from Mentor, Universal Reuse Methodology (URM) from Cadence, and Verification Methodology Manual (VMM) from Synopsys. Each one provided a base class library and TLM level modeling capabilities. Each one claimed to allow you to write reusable code.

The end result was that the customer base was confused on which one to adopt. That resulted in most people not even adopting any of the methodologies. That in turn, left almost 70% of SystemVerilog language simply unused, which wasted the effort of the originators of the language.

After intense debate on unifying all three base class libraries and methodologies, Accelera came forward and decided to pick OVM and build an industry standard methodology around it that was "universal." Most of the features of the language came from OVM, and thus UVM was born. The users felt confident in adopting this universal methodology and not get tied to a specific vendor. All vendors now support this methodology and the standard class library that comes with the tool. And oh boy, did the user community adopt it. It's **everywhere in all major companies** and with all those who develop large SoCs. It is still to proliferate (as of this writing) to companies that develop "smaller" less complex chips.

Now let us go through a high-level description of the UVM hierarchy and all the components that lie underneath it. But before that we must understand polymorphism since this technology is the bedrock of UVM.

4.2 Polymorphism

Polymorphism in its simplest form means using one object as an instance of a base class. For example, if you have the class “Creature” and from that you derive the class “elephant,” you can treat “elephant” as if it is the instance of class “Creature”.

Polymorphism is derived from Greek word, where “poly” means many and “morph” means forms, i.e., it’s an ability to appear in many forms. [(Modh)]

The OOP (object-oriented programming) term for multiple routines sharing a common name is “polymorphism.” Polymorphism allows the use of super-class handle to hold sub-class object and to access the methods of those sub-classes from the super-class handle itself.

To achieve this, functions and/or tasks in SV are declared as virtual functions/tasks to allow sub-classes to override the behavior of the function/task. So, we can say, Polymorphism = Inheritance + virtual method.

Example:

```
class vehicle; // Parent class
  virtual function vehicle_type(); // Virtual function
    $display("vehicle");
  endfunction

  virtual task color(); // Virtual task
    $display("It has color");
  endtask
endclass

class four_wheeler extends vehicle; //child class
  function vehicle_type();
    $display("It's a four wheeler");
  endfunction

  task color();
    $display("It has different colors");
  endtask
endclass

class BENZ extends four_wheeler; // "grand" child class
  function vehicle_type();
    $display("It's a BENZ");
  endfunction
```

```

    task color();
        $display("It is Black");
    endtask
endclass

program polymorphism;// program block
    initial begin
        vehicle vehcl;
        four_wheeler four_whlr;
        BENZ benz;

        four_whlr=new ();
        benz=new ();

        vehcl=four_whlr; // No need to create an object by calling a
new method
        vehcl.vehicle_type(); // accessing child class method by calling
base class method

vehcl=benz;
        vehcl.vehicle_type(); // accessing "grand" child by calling
base class method

        four_whlr=benz;
        four_whlr.color(); // accessing "grand" child method by calling
child class method
    end
endprogram

```

Output:

It's a four wheeler.

It's a BENZ.

It is black.

4.3 UVM Hierarchy

Figure 4.2 [(Accelera, Universal Verification Methodology (UVM) 1.2 User's guide)] shows a simple UVM hierarchy (aka testbench architecture). Following components make up the hierarchy.

- UVM Testbench
- UVM Test
- UVM Environment
 - The top-level environment contains one or more environments. Each environment contains an agent for a specific DUT interface. The environment may contain a scoreboard. A scoreboard compares the outputs with expected outputs.

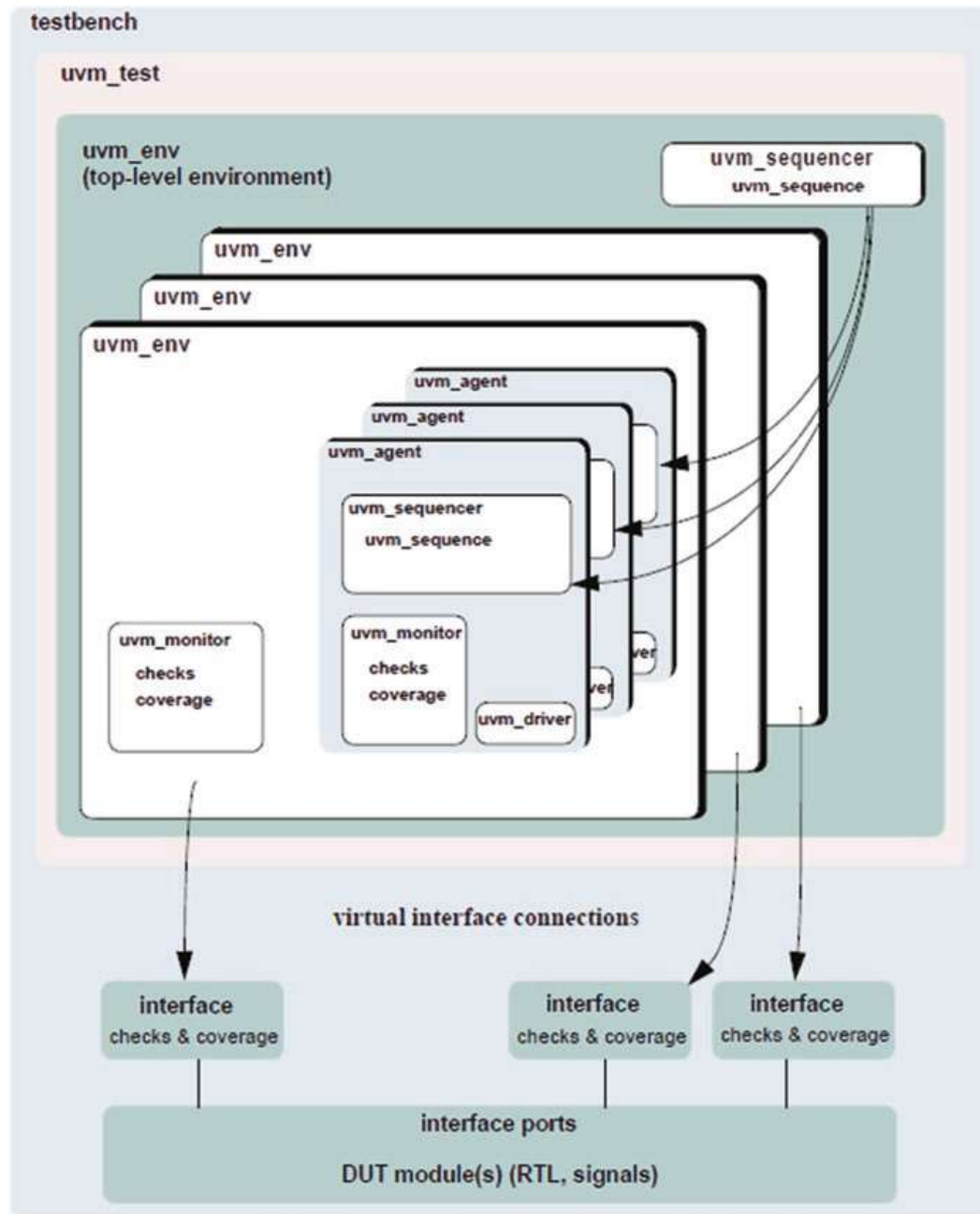


Fig. 4.2 UVM hierarchy (Acclera, Universal Verification Methodology (UVM) 1.2 User's guide)

Either the expected outputs are computed within a scoreboard or they can use (via DPI) an external reference model or any other way of comparing simulated output with expected output.

- UVM Agent
 - UVM agent comprises of a sequencer, a driver, and/or a monitor.

Let us look at each component at high level.

4.3.1 UVM Testbench

The UVM testbench typically instantiates the design under test (DUT) module and the UVM Test class and configures the connections between them. If you have module-based components, they are instantiated under the UVM testbench as well. As discussed before, TLM interfaces in UVM provide a consistent set of communication methods for sending and receiving transactions between components. The components themselves are instantiated and connected in the testbench, to perform the different operations required to verify a design.

The important point to note is that the UVM Test is dynamically instantiated at run-time, allowing the UVM testbench to be compiled once and run with many different tests.

4.3.1.1 UVM Transaction-Level Testbench

As shown in Fig. 4.3, UVM transaction-level testbench instantiates the DUT and the agent that drives the DUT. The agent comprises of the driver, the sequencer, and the monitor. An optional scoreboard to analyze data is also instantiated. This testbench is the most basic, utilizing a UVM agent. The components of this testbench are:

1. Stimulus Generator (sequencer) that creates transaction-level traffic to feed to the driver.
2. The driver then takes transactions from the sequencer, converts the transactions into pin signal-level activity, and drives the DUT.

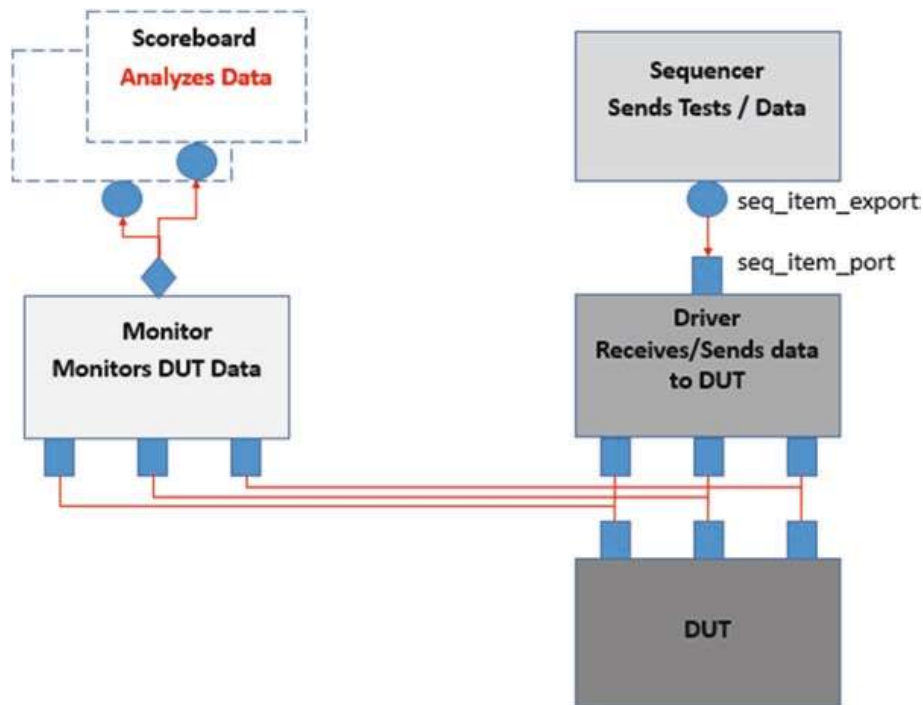


Fig. 4.3 UVM transaction-level testbench

3. A monitor that snoops the signal-level activity and converts them back into transactions that can then be fed to a scoreboard.
4. The scoreboard that gets the monitored transactions from the monitor compares them with expected transactions (response transactions).

4.3.2 UVM Test

The UVM Test is the top-level UVM component in the UVM testbench. Note that in Fig. 4.2, testbench seems to be the top-level component. But testbench merely instantiates the design under test (DUT) and the UVM Test class and configures the connection between them. A test is a class that encapsulates test-specific instructions written by the test writer.

The UVM Test typically:

1. Instantiate the top-level environment.
2. Configure the environment (via factory overrides or the configuration database).
3. Apply stimulus by invoking UVM sequences through the environment (typically one per DUT interface) to the DUT.

Tests in UVM are classes that are derived from an *uvm_test* class. Using classes allows inheritance and reuse of tests. Typically, a base test class is defined that instantiates and configures the top-level environment and is then extended to define scenario-specific configurations such as which sequences to run, coverage parameters, etc. The test instantiates the top-level environment just like any other verification component.

Typically, there is one base UVM Test with the UVM environment instantiation and other common items. Individual tests will extend this base test and configure the environment differently or select different sequences to run.

4.3.3 UVM Environment

The UVM environment is a component that groups together other verification components that are interrelated. Typical components that are usually instantiated inside the UVM environment are UVM agents, UVM scoreboards, or even other UVM environments. The top-level UVM environment encapsulates all the verification components targeting the DUT.

The top-level environment is a container that defines the reusable component topology within the UVM Tests. The top-level environment instantiates and configures the reusable verification IP and defines the default configuration of that IP as required by the application. Multiple tests can instantiate the top-level environment class and determine the nature of traffic to generate and send for the selected configuration. Additionally, the tests can override the default configuration of the top-level environment to better achieve their goals.

As noted above, the top-level UVM environment can instantiate other UVM environments. Typically for each interface to the DUT, you will find a separate environment per interface. For example, PCIe environment, USB environment, etc. Some of these IP environments can be grouped together into cluster environments (e.g., an IP interface environment, CPU environment, etc.).

4.3.4 UVM Agent

The UVM agent is a hierarchical component that groups together other verification components that are dealing with a specific DUT interface. Agent includes a UVM sequencer to manage stimulus flow, a UVM driver to apply stimulus to the DUT interface, and a UVM monitor to monitor the DUT interface. UVM agents might include other components, like coverage collectors, protocol checkers, and a TLM model.

To reiterate, the UVM agent is the component that drives the signal-level interface of the DUT. At a minimum, it will have a sequencer and a driver and an optional monitor (even though, I am not sure what you would do without a monitor to convert the DUT signal-level activity into transactions that can be sent to an analysis port to a scoreboard for pass/fail determination). The sequences are fed to the sequencer which then sends these sequences (aka tests or transactions) to the driver. The driver converts a transaction into signal-level DUT interface detail (e.g., PCIe interface). This is depicted in Fig. 4.4.

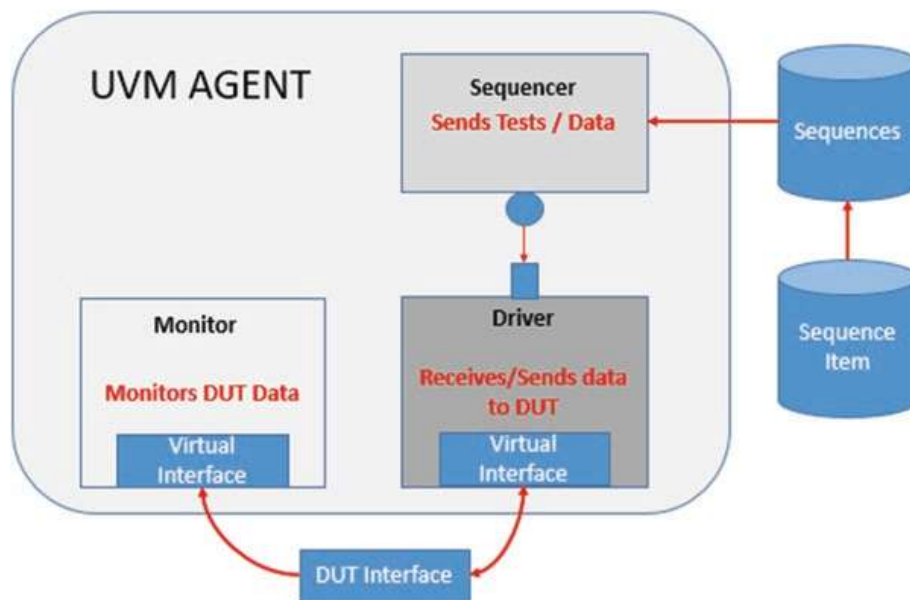


Fig. 4.4 UVM agent

Note that the agent can operate in an active mode or a passive mode. In the active mode, it can generate the stimulus (i.e., the driver drives DUT input and senses DUT outputs). In the passive mode, the driver and the sequencer remain silent (disabled) and only the monitor remains active. Monitor simply monitors the outputs of DUT; it cannot control the IO of the DUT. You can dynamically configure an agent in either an active mode or a passive mode. Monitor is an unidirectional interface, while driver is a bidirectional interface. This is depicted in Fig. 4.4.

4.3.5 UVM Sequence Item

UVM sequence item (i.e., a transaction) is the fundamental lowest denominator object in the UVM hierarchy. It is the definition of the basic transaction that will then be used to develop UVM sequences. The sequence item defines the basic transaction (e.g., an AXI transaction) data items and/or constraints imposed on them. While the driver deals with signal activities at the bit level, it doesn't make sense to keep this level of abstraction as we move away from the DUT, so the concept of transaction was created.

UVM sequence items, i.e., transactions are the smallest data transfers that can be executed in a verification model. They can include variables, constraints, and even methods for operating on themselves.

Here's an example of a sequence item:

```
class lpi_seq_item extends uvm_sequence_item;
    `uvm_object_utils(lpi_seq_item)

    //Data members
    rand bit      slp_req0;
    rand bit      slp_req1;
    rand bit      wakeup_req0;
    rand bit      wakeup_req1;
    rand bit      ss_wakeup;
    rand bit      ss_sleep;

    //UVM Constructor
    function new (string name="lpi_seq_item");
        super.new (name);
    endfunction

    //constraints on data members
    constraint slp_wakeup_reqs {
        (((slp_req0 || slp_req1) && (wakeup_req0 || wakeup_req1))
        != 1); };
endclass: lpi_seq_item
```

4.3.6 UVM Sequence

After a basic *uvm_sequence_item* has been created, the verification environment will need to generate sequences using the sequence item to be sent to the sequencer. Sequences are an ordered collection of transactions (sequence items); they shape transactions to our needs and generate as many as we want. Since the variables in the transaction (sequence item) are of type “rand,” if we want to test just a specific set of addresses in a master-slave communication topology, we could restrict the randomization to that set of values instead of wasting simulation time in invalid (or redundant) values.

Sequences are extended from *uvm_sequence*, and their main job is generating multiple transactions. After generating those transactions, there is another class that takes them to the sequencer (discussed next).

For example, using the example of sequence item shown in Sect. 4.3.5, you can write a sequence as follows.

```
class lpi_basic_seq extends uvm_sequence #(lpi_seq_item);
    `uvm_object_utils(lpi_basic_seq)

    rand int num_of_trans;

    function new (string name="lpi_basic_seq");
        super.new (name);
    endfunction

    extern task body();
endclass: lpi_basic_seq

task lpi_basic_seq::body();
    lpi_seq_item seq_item;
    seq_item = lpi_seq_item::type_id::create("seq_item");

    for (int i = 0; i < num_of_trans; i++)
    begin
        `uvm_info(get_type_name(), $psprintf("in seq for count = %d", i), UVM_LOW)

        start_item(seq_item);

        if(!seq_item.randomize())
        begin
            `uvm_error("body", "Randomization failed for seq_item")
        end

        `uvm_info(get_type_name(), $psprintf("obj is req0 = %d, req1 = %d, sleep0 = %d, sleep1 = %d", seq_item.wakeup_req0, seq_item.wakeup_req1, seq_item.slp_req0, seq_item.slp_req1), UVM_LOW)
```

```

        finish_item(seq_item);

    end
endtask: body

```

The explanation of the code is as follows:

lpi_basic_seq is extended from the *uvm_sequence* which is parameterized with the *lpi_seq_item*. We also declare a rand variable *num_of_trans*.

Then we define the task body which is the gist of the sequence. In the task *body()*, we first define a *seq_item* of type *lpi_seq_item* which is what the task body will work on.

The “for” loop will iterate for a random number of times (since *num_of_trans* is declared “rand”). It will first *start_item* (start the sequence) which is a call that *blocks* until the driver accesses the transaction being created.

Then it checks to see if *seq_item* can indeed be randomized. If not, it will generate an error and abort and will print the transaction information and do a *finish_item* which is also a blocking call which blocks until the driver has completed the operation of the current transaction.

Note that, UVM sequences can operate hierarchically with one sequence, called a parent sequence, invoking another sequence, called a child sequence. To operate, each UVM sequence is eventually bound to a UVM sequencer. Multiple UVM sequence instances can be bound to the same UVM sequencer.

4.3.7 UVM Sequencer

The sequencer controls the flow of request and response sequence items between sequences and the driver. UVM sequencer is a simple component that serves as an arbiter for controlling transaction flow from multiple stimulus sequences.

The sequencer and driver use TLM interface to communicate.

uvm_sequencer and *uvm_driver* base classes have *seq_item_export* and *seq_item_port* defined respectively. The user needs to connect them using TLM connect method.

Example:

```

driver.seq_item_port.connect(sequencer.seq_item_export);

```

Examples shown in Sects. 4.7.4 and 4.8.4 further shed light on a sequencer.

4.3.8 UVM Driver

Driver is where the TLM transaction-level world meets the DUT signal/clock/ pin-level world. Driver *receives* sequences from the sequencer, converts the received sequences into signal-level activities, and drives them on the DUT interface as per the interface protocol. Or the driver *pulls* sequences from the sequencer and sends them to the signal-level interface. This interaction will be observed and evaluated by

another block, the monitor, and as a result, the driver's functionality should only be limited to send the necessary data to the DUT. Note that nothing prevents the Driver from monitoring the transmitted/received data from DUT—but that violates the rules of modularity. Also, if you embed the monitor in the driver, you can't turn the monitor ON/OFF.

The driver has a TLM port to receive transactions from the sequencer and access to the DUT interface to drive the DUT signals.

Driver is written by extending `uvm_driver`.

`uvm_driver` is inherited from `uvm_component`; Methods and TLM port (`seq_item_port`) are defined for communication between sequencer and driver.

The `uvm_driver` is a parameterized class; and it is parameterized with the type of the request `sequence_item` and the type of the response `sequence_item`.

The UVM driver is discussed in detail with the UVM example discussed in Sect. 4.7.2.

4.3.8.1 `uvm_driver` Methods

get_next_item

This method blocks until a REQ `sequence_item` is available in the sequencer.

try_next_item

This is a non-blocking variant of the `get_next_item ()` method. It will return a null pointer if there is no REQ `sequence_item` available in the sequencer.

item_done

The non-blocking `item_done ()` method completes the driver–sequencer handshake, and it should be called after a `get_next_item ()` or a successful `try_next_item ()` call.

put

The `put ()` method is non-blocking and is used to place a RSP `sequence_item` in the sequencer.

Examples shown in Sects. 4.7.2 and 4.8.5 further shed light on a driver.

4.3.9 UVM Monitor

Monitor, in a sense, is reverse of the driver. It takes the DUT signal/pin-level activities and converts them back into transactions to be sent out to the rest of the UVM testbench (e.g., to the scoreboard) for analysis. Monitor broadcasts the created transactions through its *analysis port*. Note that comparing of the received output from the DUT to that with expected output is normally done in the scoreboard and not directly in the monitor (even though there is nothing that prevents that from happening).

The reason is to preserve modularity of the testbench. Monitor, as the name suggests, monitors the DUT signals and converts them to transactions. That's it. It's the job of the scoreboard (or any other component for that matter) to receive the broadcasted transaction from the Driver and do the comparison with the expected outputs.

The UVM monitor can perform internally some processing on the transactions produced (such as coverage collection, checking, logging, recording, etc.) or can delegate that to dedicated components connected to the monitor's analysis port.

Example shown in Sects. 4.7.5 and 4.8.6 provide examples on building a UVM monitor.

4.3.10 UVM Scoreboard

The scoreboard simply means that it is a checker (not to be confused with SystemVerilog SVA "checker"). It checks the response of the DUT against expected response. The UVM scoreboard usually receives transactions from the monitor through UVM agent analysis ports and the transactions through a reference model to produce expected transactions and then compares the expected output versus the received transaction from the monitor.

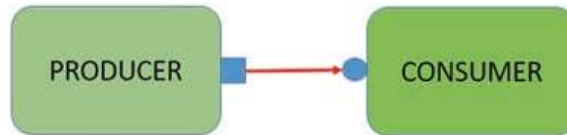
There are many ways to implement a scoreboard. For example, if you are using a reference model, you may use SystemVerilog–DPI API to communicate with the scoreboard, pass transactions via DPI to the reference model, convert reference model response into transactions, and compare the DUT output transaction with the one provided by the reference model. Reference model can be a C/C++ model or a TLM2.0 SystemC model or simply another SystemVerilog model.

4.4 UVM Class Library

Figure 4.5 [(Accelera, Universal Verification Methodology (UVM) 1.2 User's guide)] shows the building blocks of UVM class library that you can use to quickly build well constructed, reusable, configurable components and testbenches. The library contains base classes, utilities, and macros.

The advantages of using the UVM class library [(Accelera, Universal Verification Methodology (UVM) 1.2 User's guide)] include:

- (a) A robust set of built-in features—The UVM class library provides many features that are required for verification, including complete implementation of printing, copying, test phases, factory methods, and more.
- (b) Correctly implemented UVM concepts—Each component can be derived from a corresponding UVM class library component. Using these base class elements increase the readability of your code since each component's role is pre-determined by its parent class.

Fig. 4.6 UVM producer port to consumer export

The square on the producer block is called a “port,” and the circle on the consumer block is called an “export.” The producer sends out transactions and method calls (e.g., the method “put” in the following example) on its “port” (or per the language semantics, “put_port”).

Here’s sample code for a producer (Accelera, UVM 1.2 User’s Guide, 2015):

```

class producer extends uvm_component;
  uvm_blocking_put_port #(my_trans) put_port; // 1 parameter
  function new (string name, uvm_component parent);
    put_port = new ("put_port", this);
    ...
  endfunction

  virtual task run();
    my_trans myT;
    for (int i = 0; i < N; i++) begin
      // Generate myT.
      put_port.put (myT);
    end
  endtask
endclass
  
```

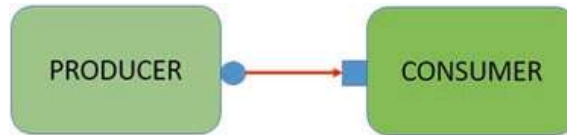
The `uvm_blocking_put_port` simply means that the producer will block until the consumer’s “put” implementation is complete. As you can see, the producer sends the transaction through its `put_port` with “put” method. This “put” method is provided as part of the `uvm_blocking_put_port` class. The “put” method is implemented by the consumer. The semantics of the put operation are defined by TLM. In this case, the `put()` call in the producer will block until the consumer’s put implementation is complete. Other than that, the operation of producer is completely independent of the put implementation (`uvm_blocking_put_imp`). The modularity provided by TLM fosters an environment in which components may be easily reused since the interfaces are well defined.

Here’s an implementation of the consumer.

```

class consumer extends uvm_component;
  uvm_blocking_put_imp #(my_trans, consumer) put_export; // 2
  parameters
  ...
  task put (my_trans t);
    case (t.kind)
  
```

Fig. 4.7 Producer export to consumer port



```

        BURST_READ: // Do burst read.
        BURST_WRITE: // Do burst write.
    endcase
endtask
endclass

```

To reiterate, whenever the producer invokes the “put” method, the consumer’s implementation (task *put*) will be executed.

Converse of “put” is “get” (Fig. 4.7). In other words, here the consumer *gets* a transaction from the producer. The consumer requests transactions from the producer via its “get” port. Note that now the “square box” (i.e., the *port*) is on the Consumer and the circle (i.e., the *export*) is on the producer side. This means that consumer calls the “get” method which is implemented by the producer.

Here’s a simple example (Accelera, Universal Verification Methodology (UVM) 1.2 User’s guide).

```

class get_consumer extends uvm_component;
    uvm_blocking_get_port #(my_trans) get_port;
    function new (string name, uvm_component parent);
        get_port = new ("get_port", this);
        ...
    endfunction

    virtual task run();
    my_trans myT;
        for (int i = 0; i < N; i++) begin
            // Generate t.
            get_port.get(myT);
        end

    endtask
endclass

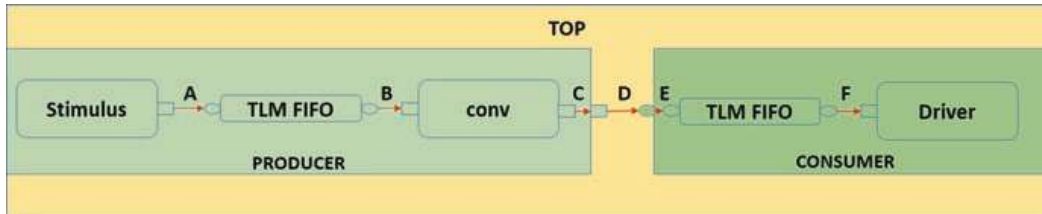
```

As with *put()*, the *get_consumer*’s *get()* call will block until the *get_producer*’s method completes. In TLM terms, *put()* and *get()* are blocking methods.

```

class get_producer extends uvm_component;
    uvm_blocking_get_imp #(my_trans, get_producer) get_export;
    ...
    task get(output my_trans t);

```

Fig. 4.8 Producer to consumer via TLM FIFO**Fig. 4.9** Stimulus to driver hierarchical connection

```

        simple_trans tmp = new();
        // Assign values to tmp.
        t = tmp;
    endtask
endclass

```

Note: In both these examples, there is a single process running, with control passing from the port to the export and back again. The direction of data flow (from producer to consumer) is the same in both examples.

But what if you want the producer and the consumer to operate independently? In the example above, the consumer will be active only when its `put()` method is called by the producer. UVM provides the `uvm_tlm_fifo` channel to facilitate such communication. The `uvm_tlm_fifo` implements all the TLM interface methods, so the producer *puts* the transaction into the `uvm_tlm_fifo`, while the consumer independently *gets* the transaction from the FIFO, as shown in Fig. 4.8.

When the producer puts a transaction into the FIFO, it will block if the FIFO is full; otherwise, it will put the object into the FIFO and return immediately. The `get` operation will return immediately if a transaction is available (and will then be removed from the FIFO); otherwise, it will block until a transaction is available. Thus, two consecutive `get()` calls will yield different transactions to the consumer. The related `peek()` method returns a copy of the available transaction without removing it. Two consecutive `peek()` calls will return copies of the same transaction.

4.5.2 Hierarchical Connections

Ok, let us take the above examples a bit further. What if you want to make connections across hierarchical boundaries? Let us consider the hierarchical design shown in Fig. 4.9.

Here again you have the producer and the consumer. But there is a hierarchy under each of these components. The idea is to transfer a “stim” transaction (stimulus transaction) to the eventual “drv” (driver) component.

The producer contains “stim,” “fifo,” and “conv,” while the consumer contains “fifo” and “drv”.

Note that just as we saw in previous examples, the producer has a put_port (i.e., the port puts a transaction) and the consumer has a put_export (i.e., it implements the method called by the producer).

Connections C and E are of a different sort than what have seen so far. Connection C is a port-to-port connection, and connection E is an export-to-export connection. These two kinds of connections are necessary to complete hierarchical connections. Connection C imports a port from the outer component to the inner component. Connection E exports an export upwards in the hierarchy from the inner component to the outer one. Ultimately, every transaction-level connection must resolve so that a port is connected to an export. However, the port and export terminals do not need to be at the same place in the hierarchy. We use port-to-port and export-to-export connections to bring connectors to a hierarchical boundary to be accessed at the next higher level of hierarchy.

4.5.3 Analysis Ports and Exports

Ok so far, we have seen “port” and “export.” Let us look at a third type of interface called “analysis_port.” This port is represented as a *diamond* (Fig. 4.10). As we discussed above, the key distinction between the two types of TLM communication is that the put/get ports typically require a corresponding export to supply the implementation. For analysis, however, the emphasis is on a particular component, such as a monitor, being able to produce a stream of transactions, regardless of whether there is a target actually connected to it. Modular analysis components are then connected to the analysis_port, each of which processes the transaction stream in a particular way.

Let us look at the analysis port and export separately.

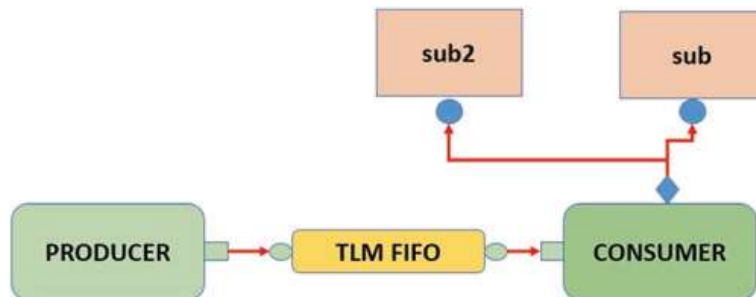


Fig. 4.10 Analysis port

In UVM semantics, the analysis port is called “`uvm_analysis_port`.” It’s a specialized TLM port whose interface consists of a single function called “`write()`.” The analysis port contains a list of `analysis_exports` that are connected to it. Refer to Fig. 4.10. When the component calls `analysis_port.write()`, the `analysis_port` cycles through the list of connected exports and calls the `write()` method of each connected export. If nothing is connected, the `write()` call simply returns. Thus, an analysis port may be connected to zero or many analysis exports, but the operation of the component that writes to the analysis port does not depend on the number of exports connected. Because `write()` is a void function, the call will always complete in the same delta cycle, regardless of how many components (for example, scoreboards, coverage collectors, and so on) are connected.

Components to which *export* is connected implement the `write()` function. If multiple exports are connected to an analysis port, the port will call the `write()` of each export, in order. Since all implementations of `write()` must be functions, the analysis port’s `write()` function completes immediately, regardless of how many exports are connected to it. When multiple subscribers are connected to an `analysis_port`, each is passed a pointer to the same transaction object, the argument to the `write()` call. Each `write()` implementation must make a local copy of the transaction and then operate on the copy to avoid corrupting the transaction contents for any other subscriber that may have received the same pointer. UVM also includes an `analysis_fifo`, which is a `uvm_tlm_fifo` that also includes an analysis export, to allow blocking components access to the analysis transaction stream. The `analysis_fifo` is unbounded, so the monitor’s `write()` call is guaranteed to succeed immediately. The analysis component may then get the transactions from the `analysis_fifo` whenever it pleases.

4.6 UVM Phases

As shown in Fig. 4.11, following phases make up the UVM Phases. Each phase is described below. Basically, there are three phases overall. The build phase builds top-level testbench topology and the connect phase connects environment topology. The run phase does exactly what the name suggests, namely, run the test! All phases “under” the run umbrella run in zero time, except of course the `run()` phase. And finally, the cleanup phase gathers details on the final DUT state, processes the simulation results, and does simulation results analysis and reporting.

The following provides at finer granularity the phases at play in UVM.

4.6.1 Build Phases

The build phases are executed at the start of the UVM testbench simulation, and their overall purpose is to construct, configure, and connect the testbench component hierarchy [(MentorGraphics)].

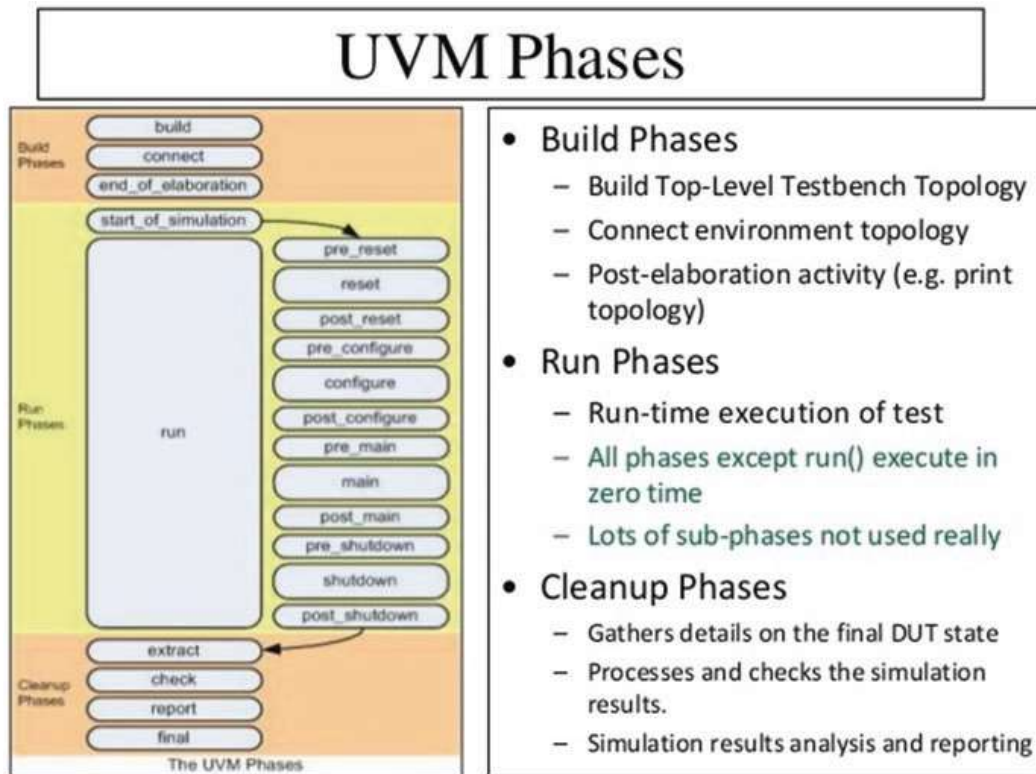


Fig. 4.11 UVM phases (MentorGraphics)

All the build phase methods are functions and therefore execute in zero simulation time.

4.6.1.1 Build

Once the UVM testbench root node component is constructed, the build phase starts to execute. It constructs the testbench component hierarchy from the top of the hierarchy downwards. The construction of each component is deferred so that each layer in the component hierarchy can be configured by the level above. During the build phase, uvm_components are indirectly constructed using the UVM factory.

4.6.1.2 Connect

The connect phase is used to make TLM connections between components or to assign handles to testbench resources. It must occur after the build method has put the testbench component hierarchy in place and works from the bottom of the hierarchy upwards.

4.6.1.3 end_of_elaboration

The end_of_elaboration phase is used to make any final adjustments to the structure, configuration, or connectivity of the testbench before simulation starts. Its implementation can assume that the testbench component hierarchy and interconnectivity is in place. This phase executes bottom up.

4.6.2 Run-Time Phases

The testbench stimulus is generated and executed during the run-time phases which follow the build phases. After the start_of_simulation phase, the UVM executes the run phase and the phases pre_reset through to post_shutdown in parallel. The run phase was present in the OVM and is preserved to allow OVM components to be easily migrated to the UVM. The other phases were added to UVM to give finer run-time phase granularity for tests, scoreboards, and other similar components. It is expected that most testbenches will only use reset, configure, main, and shutdown and not their pre- and post variants.

4.6.2.1 start_of_simulation

The start_of_simulation phase is a function which occurs before the time-consuming part of the testbench begins. It is intended to be used for displaying banners, testbench topology, or configuration information. It is called in bottom-up order.

4.6.2.2 Run

The run phase occurs after the start_of_simulation phase and is used for the stimulus generation and checking activities of the testbench. *The run phase is implemented as a task, and all uvm_component run tasks are executed in parallel.* Transactors such as drivers and monitors will nearly always use this phase.

4.6.2.3 pre_reset

The pre_reset phase starts at the same time as the run phase. Its purpose is to take care of any activity that should occur before reset, such as waiting for a power-good signal to go active. I do not anticipate much use for this phase.

4.6.2.4 Reset

The reset phase is reserved for DUT or interface-specific reset behavior. For example, this phase would be used to generate a reset and to put an interface into its default state.

4.6.2.5 post_reset

The post_reset phase is intended for any activity required immediately following reset. This might include training or rate negotiation behavior. I do not anticipate much use for this phase.

4.6.2.6 pre_configure

The pre_configure phase is intended for anything that is required to prepare for the DUT's configuration process after reset is completed, such as waiting for components (e.g., drivers) required for configuration to complete training and/or rate negotiation. It may also be used as a last chance to modify the information described by the test/environment to be uploaded to the DUT. I do not anticipate much use for this phase.

4.6.2.7 Configure

The configure phase is used to program the DUT and any memories in the testbench so that it is ready for the start of the test case. It can also be used to set signals to a state ready for the test case start.

4.6.2.8 post_configure

The post_configure phase is used to wait for the effects of configuration to propagate through the DUT or for it to reach a state where it is ready to start the main test stimulus. I do not anticipate much use for this phase.

4.6.2.9 pre_main

The pre_main phase is used to ensure that all required components are ready to start generating stimulus. I do not anticipate much use for this phase.

4.6.2.10 Main

This is where the stimulus specified by the test case is generated and applied to the DUT. It completes when either all stimulus is exhausted or a time-out occurs. Most data throughput will be handled by sequences started in this phase.

4.6.2.11 post_main

This phase is used to take care of any finalization of the main phase. I do not anticipate much use for this phase.

4.6.2.12 pre_shutdown

This phase is a buffer for any DUT stimulus that needs to take place before the shutdown phase. I do not anticipate much use for this phase.

4.6.2.13 Shutdown

The shutdown phase is used to ensure that the effects of stimulus generated during the main phase have propagated through the DUT and that any resultant data has drained away.

4.6.2.14 post_shutdown

Perform any final activities before exiting the active simulation phases. At the end of the post_shutdown phase, the UVM testbench execution process starts the cleanup phases. I do not anticipate much use for this phase.

4.6.3 Cleanup Phases

The cleanup phases are used to extract information from scoreboards and functional coverage monitors to determine whether the test case has passed and/or reached its coverage goals [(MentorGraphics)]. The cleanup phases are implemented as functions and therefore take zero time to execute. They work from the bottom upwards in the component hierarchy.

4.6.3.1 Extract

The extract phase is used to retrieve and process information from scoreboards and functional coverage monitors. This may include the calculation of statistical information used by the report phase. This phase is usually used by analysis components.

4.6.3.2 Check

The check phase is used to check that the DUT behaved correctly and to identify any errors that may have occurred during the execution of the testbench. This phase is usually used by analysis components.

4.6.3.3 Report

The report phase is used to display the results of the simulation or to write the results to file. This phase is usually used by analysis components.

4.6.3.4 Final

The final phase is used to complete any other outstanding actions that the testbench has not already completed.

Here's a very simple example of a basic *uvm_component* showing different UVM phases.

```
class generic_component extends uvm_component;
    `uvm_component_utils(generic_component)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction: new

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

        //Code for constructors goes here
    endfunction: build_phase

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);

        //Code for connecting components goes here
    endfunction: connect_phase
```

```

    task run_phase(uvm_phase phase);
        //Code for simulation goes here
    endtask: run_phase

    function void report_phase(uvm_phase phase);
        //Code for showing simulation results goes here
    endfunction: report_phase

endclass: generic_component

```

4.7 UVM Example: One

We now present a UVM example, which describes building sequence_item, sequence, UVM agent, UVM sequencer, UVM driver, etc.

4.7.1 Modeling a Sequence Item

A sequence item is essentially a transaction upon (using) which sequences can be built. So, they are transaction objects used as stimulus to the DUT. The UVM class library provides the uvm_sequence_item base class. Every user-defined sequence item should be derived directly or indirectly from this base class.

UVM has built-in automation for many service routines that a data item needs. For example, you can use:

- Print() to print a data item
- Copy() to copy the contents of a data item
- Compare() to compare two similar objects

UVM allows you to use a built-in, mature, and consistent implementation of these routines.

Here's an example.

```

class bus_seq_item extends uvm_sequence_item;

    // Request data properties are rand
    rand logic [31:0] addr;
    rand logic [31:0] write_data;
    rand bit read_not_write;
    rand int delay;

    // Response data properties are NOT rand
    bit error;
    logic [31:0] read_data;

```